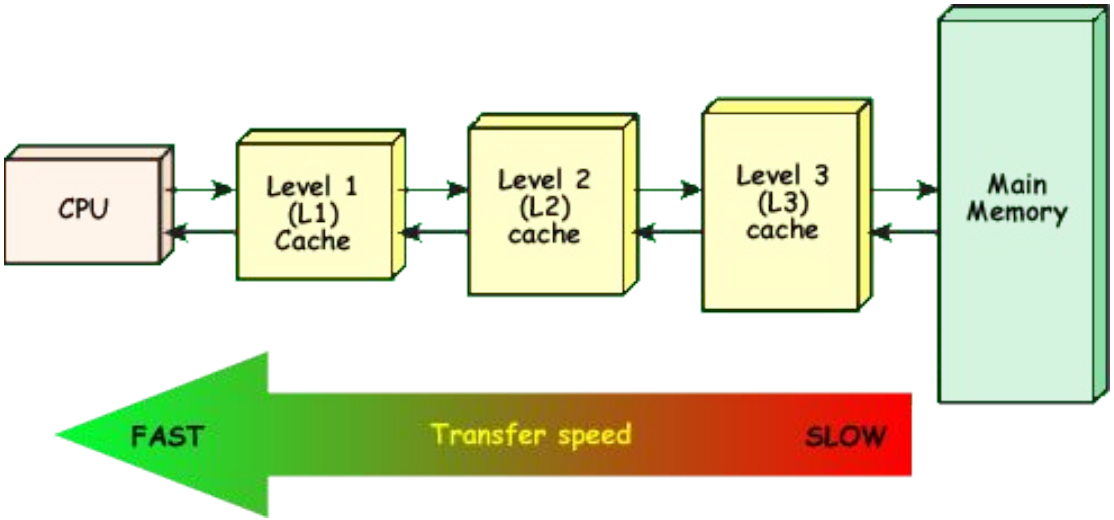
Data Structures

Lecture 6
Types of Linked List + Doubly Linked List Functions

Prantik Paul [PNP]
Lecturer
Department of Computer Science and Engineering
BRAC University

Cost of Traversal (Array vs List)



Singly Linked List (Problems)

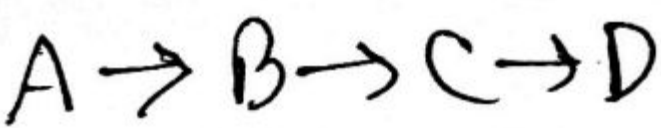
1. Traversal
2. Determine where to stop
3. Operations near the end
4. Moving back in Time

Types of Linked List

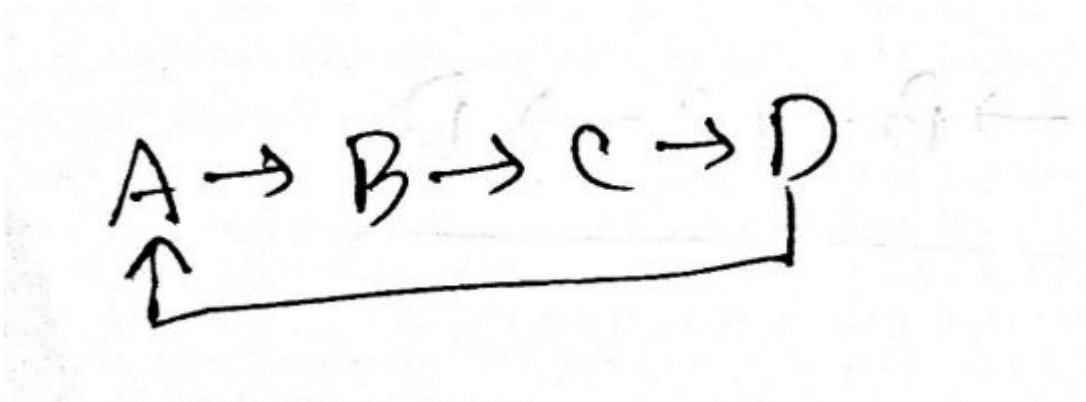
Head Type	Connection	Structure

Category 1	Category 2	Category 3
Non-Dummy Headed	Singly	Linear
Dummy Headed	Doubly	Circular

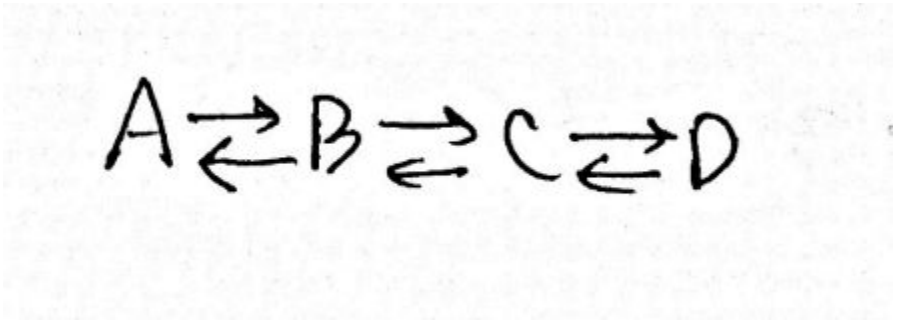
Non-Dummy Headed Singly Linear Linked List



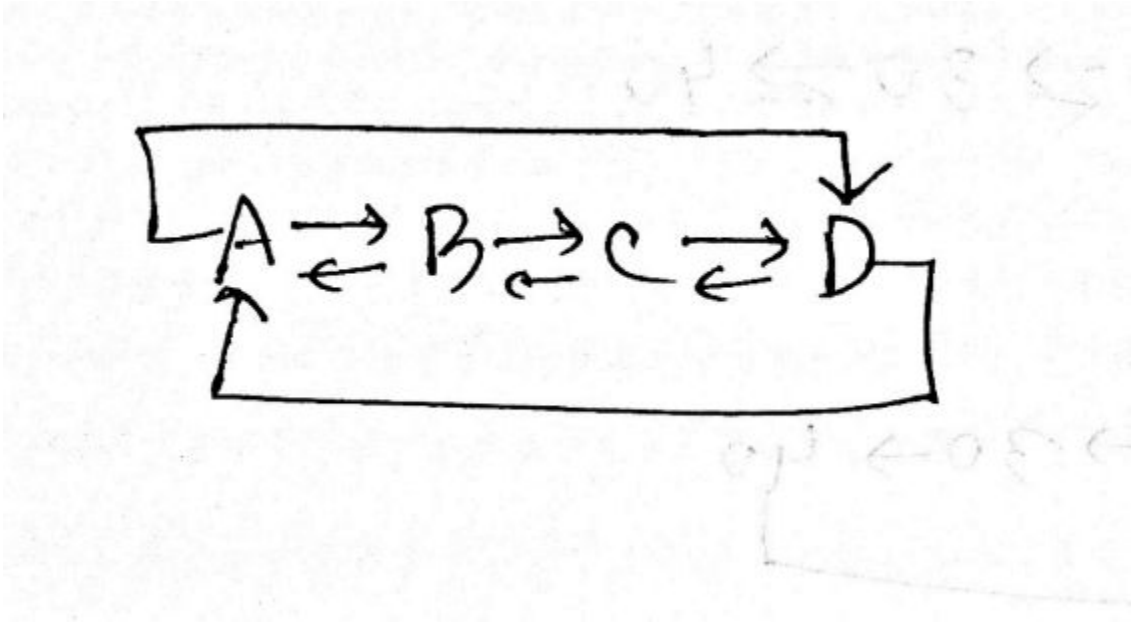
Non-Dummy Headed Singly Circular Linked List



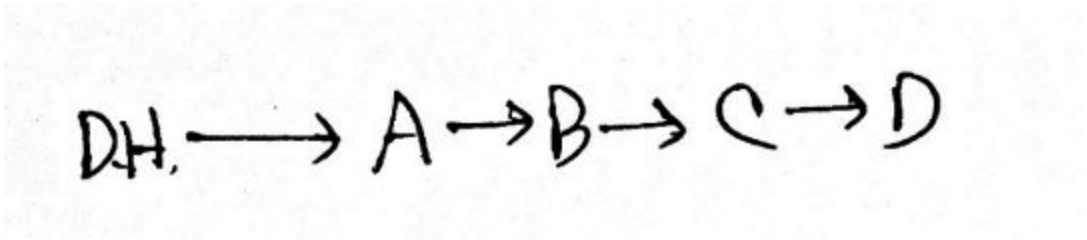
Non-Dummy Headed Doubly Linear Linked List



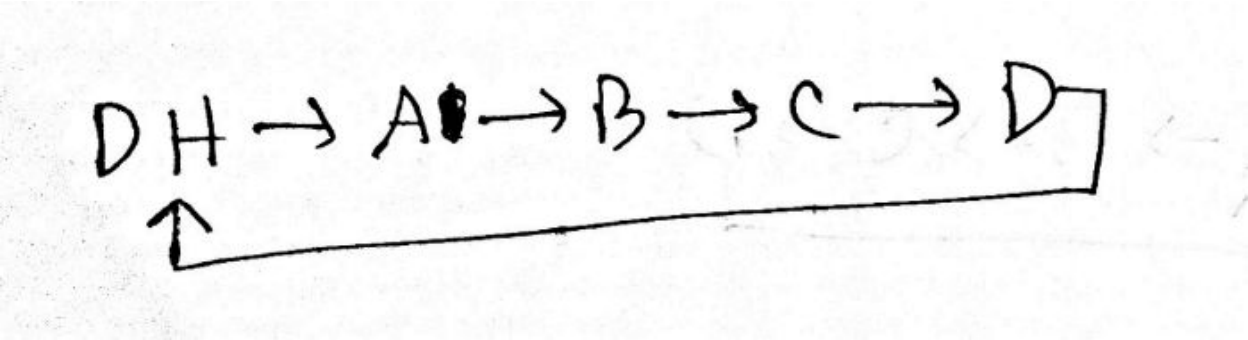
Non-Dummy Headed Doubly Circular Linked List



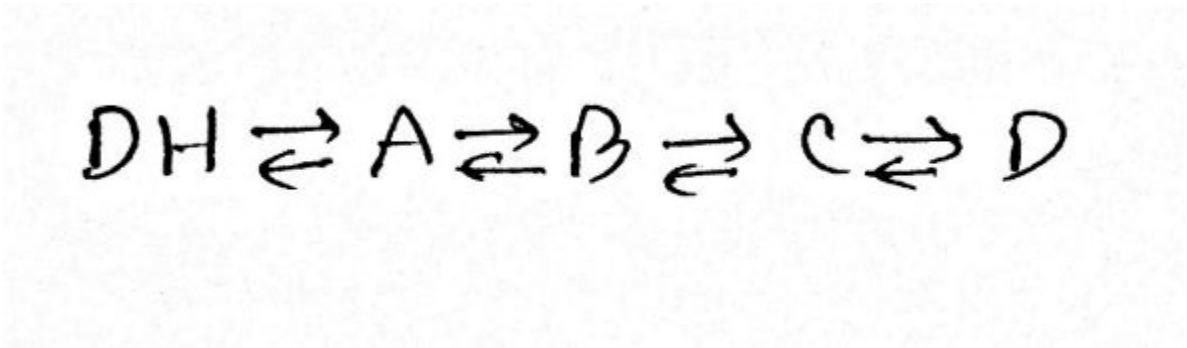
Dummy Headed Singly Linear Linked List



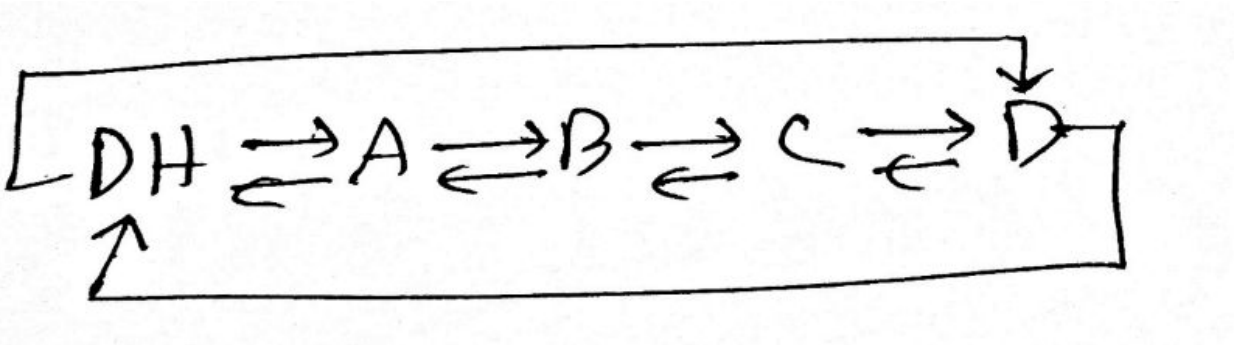
Dummy Headed Singly Circular Linked List



Dummy Headed Doubly Linear Linked List



Dummy Headed Doubly Circular Linked List



Doubly Linked List (Node Class)

```
class DoublyNode:
    def __init__(self, elem, next, prev):
        self.elem = elem
        self.next = next # To store the next node's reference.
        self.prev = prev # To store the previous node's reference.
```

Doubly Linked List (Creation)

```
1  def createList(a):
2      dh = DoublyNode(None, None, None)
3      dh.next = dh
4      dh.prev = dh
5      tail = dh
6
7      for i in range(len(a)):
8          n = DoublyNode(a[i], dh, tail)
9          tail.next = n
10         tail = tail.next
11         dh.prev = tail
12
13     return dh
```

Doubly Linked List (Iteration)

```
1 def iteration(dh):
2     temp = dh.next
3     while temp != dh:
4         print(temp.elem)
5         temp = temp.next
```

Doubly Linked List (GetNode)

```
1 def nodeAt(dh, idx):
2     temp = dh.next
3     c = 0
4     while temp != dh:
5         if c == idx:
6             return temp
7         c += 1
8         temp = temp.next
9     return None # Invalid Index
```

Doubly Linked List (Insertion)

```
1 def insertion(dh, elem, idx):
2   # Assuming the idx is valid
3   node_to_insert = DoublyNode(elem, None, None)
4   indexed_node = nodeAt(dh, idx) # Retriving the node at that index
5   prev_node = indexed_node.prev # There will always be a previous node
6   # Change the connection
7   # Observe that no special case is needed
8   node_to_insert.next = indexed_node
9   node_to_insert.prev = prev_node
10  prev_node.next = node_to_insert
11  indexed_node.prev = node_to_insert
```

Doubly Linked List (Removal)

```
1  def removal(dh, idx):
2      # Assuming the idx is valid
3      node_to_remove = nodeAt(dh, idx)
4      prev_node = node_to_remove.prev
5      next_node = node_to_remove.next
6      # Change the connection
7      # No special case is needed
8      prev_node.next = next_node
9      next_node.prev = prev_node
10     node_to_remove.next = None
11     node_to_remove.prev = None
12     return node_to_remove.elem # Returning the removed element
```